

MEDIATEK

MT8676 Android OTA User Manual

Version: 1.0

Release Date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Ming Ji	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	4
1 OTA	5
1.1 Overview	5
1.1.1 Purpose	5
1.1.2 Who Should Read This Document	5
1.1.3 How to Use This Manual	5
1.2 References	6
1.3 OTA Update Definitions	6
1.4 OTA Abbreviations	7
1.5 Enable A/B System Updates	7
1.6 Architecture Overview	7
1.6.1 Preloader Boot Control Flow	8
1.6.2 A/B System Partition Layout	8
1.6.3 Ramdisk Location in A/B System Updates	9
1.6.4 Boot Control HAL and HIDL	10
1.6.5 A/B System Updates Process	12
1.6.6 Virtual A/B	12
1.7 Guideline for Generating A/B OTA Package	13
1.7.1 Generate A/B OTA Update Package by Split Build 1.0 (LD1.0) Step by Step	13
1.7.2 Generate A/B OTA Update Package by Split Build 2.0 (LD2.0) Step by Step	15
1.8 A/B System Updates Limitations	17
1.9 How to Use and Debug A/B System Updates	17
1.9.1 How to Use A/B Updates	17
1.9.2 How to Debug A/B Updates	18
1.10 Frequently Asked Questions	18
1.10.1 Does flashtool Need to Download Both A and B Images	18
1.10.2 How to Know What Image is Running (A or B)	18
1.10.3 Could A Partition Load B Partition (e.g., lk_a loads boot_b)	19
1.10.4 What Partition Needs A/B	19
1.10.5 What's Feature Option for A/B System	19
1.10.6 If Handset Ships without A/B, Could Handset Become A/B through OTA Update	19
1.10.7 Who Set boot_sucessfully Flag	19
Exhibit 1 Terms and Conditions	20

List of Figures

Figure 1-1. Preloader boot control flow	8
---	---

Lisf of Tables

Table 1-1. Chapter overview5

Table 1-2. Abbreviations.....7

1 OTA

1.1 Overview

The introduction of A/B System updates is as follows:

- It is a new Over-The-Air (OTA) feature alias Seamless Updates .
- No more bricks (can fall back to previous system image).
- Ensures that a workable booting system remains on the disk.
- OTA updates applied in the normal mode background.



1.1.1 Purpose

This document provides the user guidelines for the A/B System updates. It describes how to generate the OTA package on the Android platform. This manual also elaborates the mechanism required to enable A/B System updates (Seamless Updates) feature in MTK platform.

1.1.2 Who Should Read This Document

This document is primarily intended for:

- Engineers with technical knowledge of the OTA A/B System updates (Seamless Updates)

1.1.3 How to Use This Manual

This segment explains how information is distributed in this document and presents some cues and examples to simplify finding and understanding information in this document. [Table 1-1](#) presents an overview of the chapters and appendices in this document.

Table 1-1. Chapter overview

#	Chapter	Contents
1	Introduction	Describes the scope and layout of this document.
2	References	Provides reference website or documents

#	Chapter	Contents
3	OTA Update Definitions	Presents OTA update definitions
4	Abbreviations	Lists OTA abbreviations
5	Enable A/B System Updates	Shows how to enable A/B System updates in MediaTek platform
6	Architecture Overview	Demonstrates A/B System updates architecture overview
7	Guideline for Generating A/B OTA Package	Describes how to generate an incremental and full OTA package
8	A/B System Updates Limitations	Describes A/B System updates limitations
9	How to Use and Debug A/B System Updates	Explains how to use and debug A/B System updates
10	Frequently Asked Questions	Lists frequently asked questions and answers

1.2 References

The following documents contain provisions which, through reference in this text, constitute provisions of the present document.

- [1] The Android OTA Package Tools: <https://source.android.com/devices/tech/ota/tools.html>
- [2] A/B (Seamless) System updates:
https://source.android.com/devices/tech/ota/ab_updates
https://source.android.com/devices/tech/ota/inside_packages.html
- [3] Implementing A/B Updates: https://source.android.com/devices/tech/ota/ab_implement
- [4] Virtual A/B Updates: https://source.android.com/docs/core/ota/virtual_ab
- [5] Configure ART: <https://source.android.com/devices/tech/dalvik/configure>
- [6] Ramdisk storage location: <https://source.android.com/docs/core/architecture/partitions/generic-boot>

1.3 OTA Update Definitions

For the purposes of the present document, the following terms and definitions apply:

This chapter is included from [1].

Full updates: A full update is one where the entire final state of the device (system, boot, and recovery partitions) is contained in the package. As long as the device is capable of receiving the package and booting the recovery system, **the package can install the desired build regardless of the current state of the device.**

Incremental updates: An incremental update contains a set of binary patches to be applied to the data already on the device. This can result in considerably smaller update packages:

- Files that have not changed don't need to be included.
- Files that have changed are often very similar to their previous versions, so the package need only contain an encoding of the differences between the two files.

Note: You can install the incremental update package only on a device that has the old or source build used when constructing the package.

Virtual A/B: Google introduces a new feature named Virtual A/B that can use COW device created by device mapper to save super partition size.

1.4 OTA Abbreviations

Please note the abbreviations and their explanations provided in [Table 1-2](#). They are used in many fundamental definitions and explanations in this document and are specific to the information that this document contains.

Table 1-2. Abbreviations

Abbreviations	Explanation
LK	Little Kernel
OTA	Over-The-Air

1.5 Enable A/B System Updates

Please set **MTK_AB_OTA_UPDATER** to **yes** in the following files (if MTK_AB_OTA_UPDATER does not exist in the corresponding files, please add this configuration in the files):

- **ProjectConfig.mk:**
 - device/mediatekprojects/<project_name>/ProjectConfig.mk
- **Preloader:**
 - vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/<project_name>/<project_name>.mk
- **SystemConfig.mk:**
 - device/mediatek/system/<mssi_xxxx> /SystemConfig.mk

If the project is using LK:

- **LK:**
 - vendor/mediatek/proprietary/bootable/bootloader/lk/project/<project_name>.mk

If the project is using LK2:

- **LK2:**
 - vendor/mediatek/proprietary/bootable/bootloader/lk2/project/<project_name>.mk

1.6 Architecture Overview

This section gives a brief description of the modules of the system and the relationship of the modules.

1. Preloader boot control flow
2. LK boot control flow
3. A/B System partition layout
4. Ramdisk location in A/B System updates
5. Boot control HAL and HIDL

6. DM-verity for A/B System updates
7. Fastboot for A/B System updates
8. A/B system update process

1.6.1 Preloader Boot Control Flow

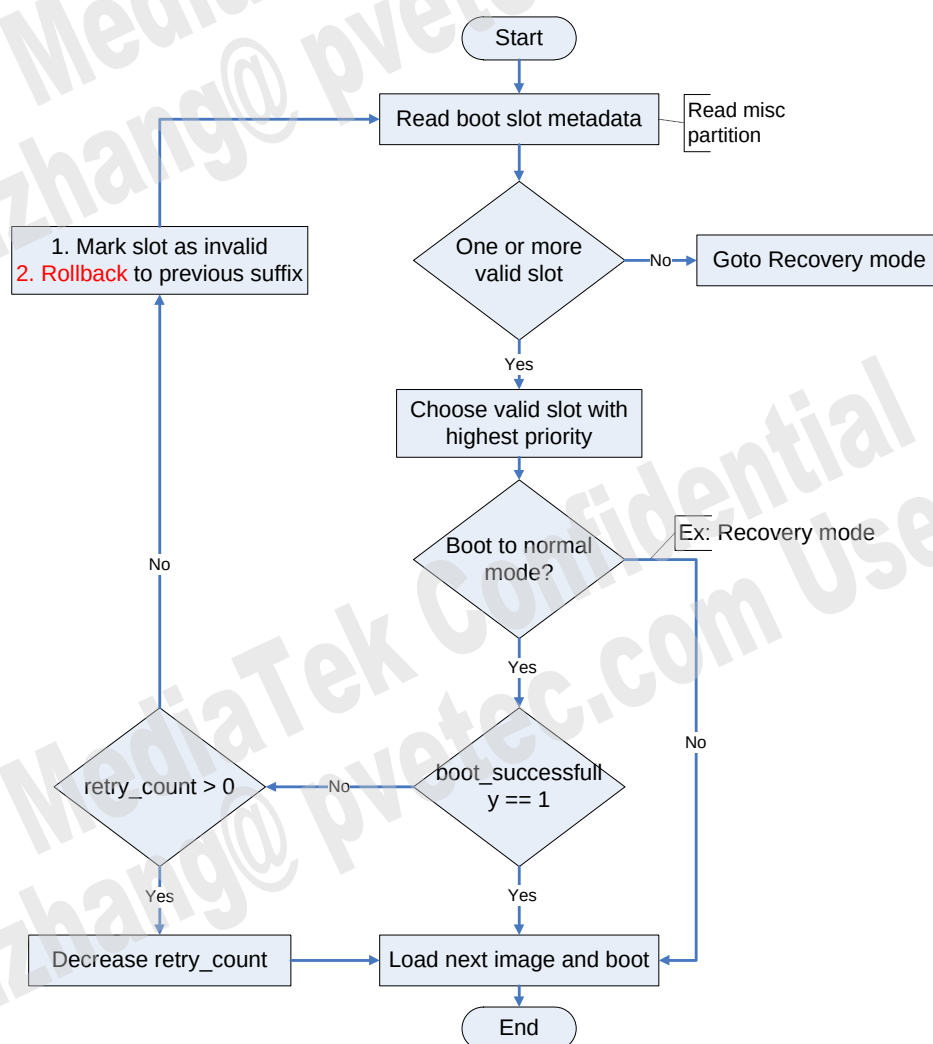
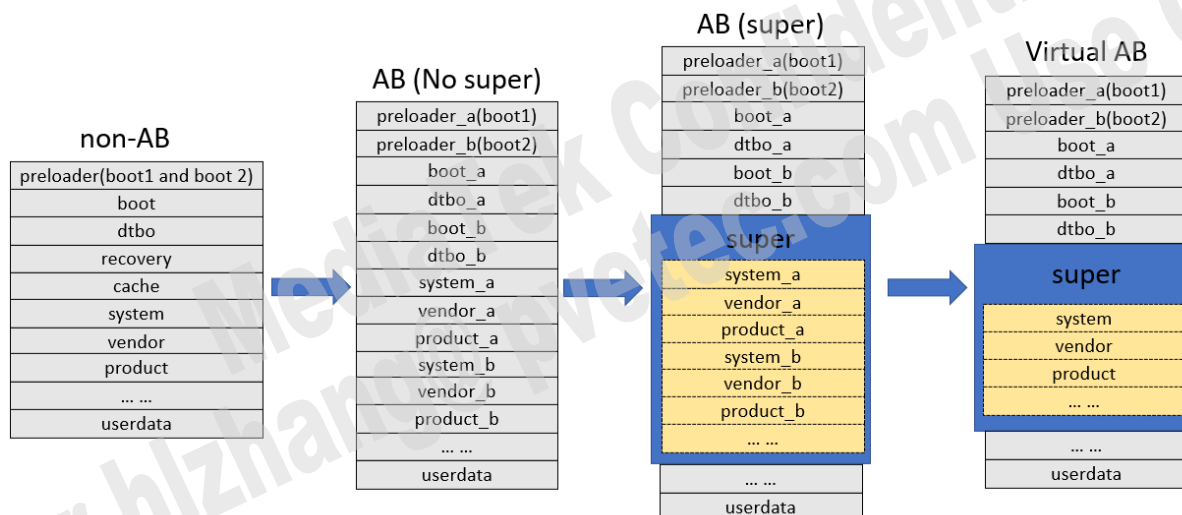


Figure 1-1. Preloader boot control flow

1.6.2 A/B System Partition Layout

The partition layout for various types of systems is as follows:



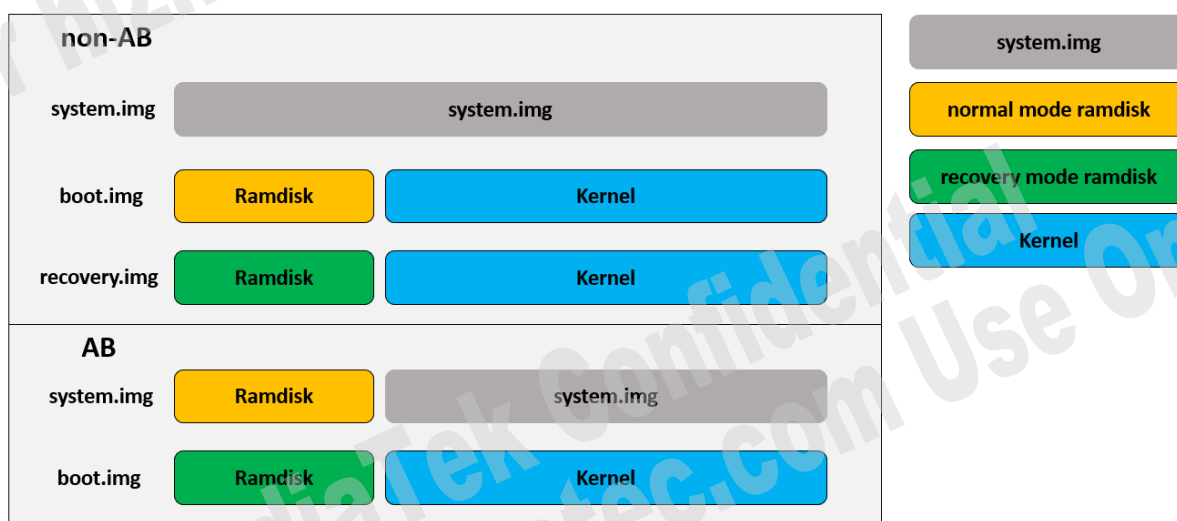
The partition layout differences of various systems are as follows:

- AB system no longer needs recovery and cache partitions.
- Starting from Android Q, Google has added a dynamic partitioning function to put system, vendor, product and other partitions into super partitions.
- Starting from Android R, Google has added the virtual A/B function. During operation, the system will only save a copy of the image information of super sub-partitions such as systems, vendor, and products, saving the storage space occupied by super partitions.

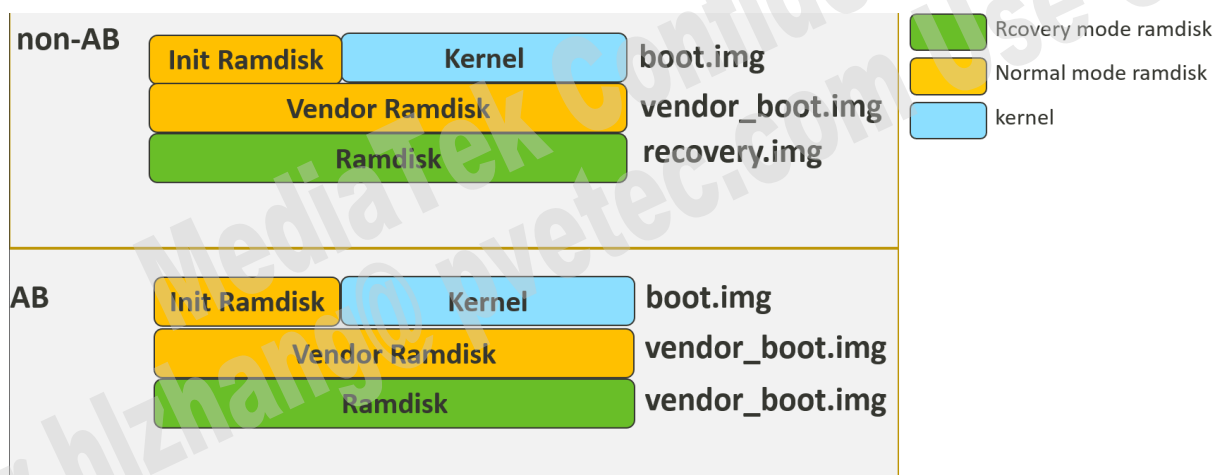
1.6.3 Ramdisk Location in A/B System Updates

There are differences in the storage locations of ramdisk information in different Android versions. Please refer to Google instructions for details: <https://source.android.com/docs/core/architecture/partitions/generic-boot>.

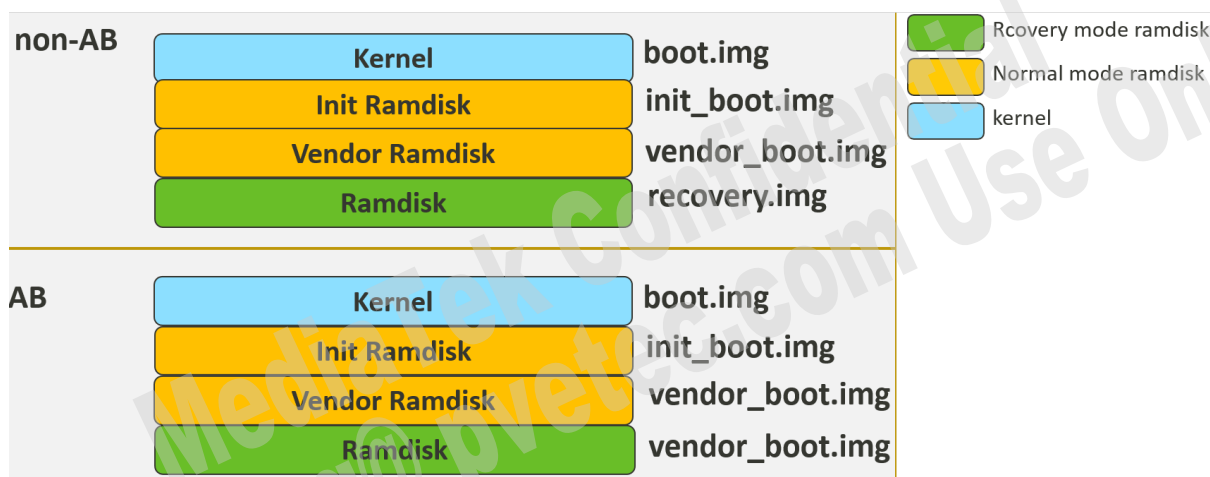
1. If the system Android version is smaller than Android R, the location of the ramdisk is as follows:



2. If the system Android version is Android R or Android S, the location of the ramdisk is as follows:



3. Starting from Android T, the location of the ramdisk is as follows:



1.6.4 Boot Control HAL and HIDL

MediaTek uses **AOSP default boot control HIDL** interface to access boot control HAL

Source code path of bootctrl is `hardware/interfaces/boot`

Boot control HAL for update_engine

- Android version $\geq$ U
 - Source code path of bootctrl is `vendor/mediatek/proprietary/hardware/bootctrl_service/`
- Android version $\geq$ R
 - Source code path of bootctrl is `vendor/mediatek/proprietary/hardware/bootctrl/`

Preloader (PL) and LK also have boot control API

- PL:**
 - `vendor/mediatek/proprietary/bootable/bootloader/preloader/platform/common/bootctrl/`
- LK:**
 - `vendor/mediatek/proprietary/bootable/bootloader/lk/platform/common/bootctrl/1.0`

When the device first boots, the preloader sets the **default values** for the launch control structure.

Default value in misc partition	
slot A	slot B
priority = 15 tries_remaining= 7 successful_boot = 1	priority = 15 tries_remaining = 7 successful_boot = 0

After device’s first OTA update, it will boot from B, because slot_B has **higher priority** than slot_A.

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 7 successful_boot = 0

After the device is successfully booted from slot_B for the first time, the successful_boot of slot_B will be set to 1.

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 5 successful_boot = 1

After the device is successfully booted from slot_B for the second time, the tries_remaining of slot_B will be set to 1.

Default value in misc partition	
slot A	slot B
priority = 14 tries_remaining = 1 successful_boot = 1	priority = 15 tries_remaining = 1 successful_boot = 1

1.6.5 A/B System Updates Process

- **Update_engine:**
 - Executed by update_engine.rc
 - Only responsible for update partition like role of recovery and updater
 - Uses payload.bin which is in update.zip to update each partition
 - Runs in little core and background
 - Calls boot_control HAL/HIDL interface
 - Runs a postinstall program from the new partition after writing all the unused slot partitions

Update_engine should **NOT** do:

- Modifies the partition table (Layout changed)
 - Modifies the contents of partitions in the current slot
- **Update_engine_client:**
 - Update_engine_client is for debug only, this process could get update status.
 - **Update_verifier:**
 - Update_verifier is executed by update_verifier.rc
 - If the product supports verity, check whether verity mode is enforcing mode or not
 - Mark **boot successful flag** when the platform boots up

1.6.6 Virtual A/B

- Virtual A/B is introduced in Android R that can use COW device to save super partition size. But after the update flow is done, it needs another system reboot step to merge the complete super partition content.
- **Virtual A/B is a GMS requirement on devices launching with Android 11 and higher.**
- **Limitation:**
 - Sideload update under recovery for incremental package is no longer supported.
 - The following note is from Google VAB document and it depicts the detail of the limitation.
The userdata partition cannot be mounted under recovery if it is encrypted. Free Blocks within the userdata partition cannot be determined, so we cannot allocate COW files on the userdata partition. Hence, we cannot create new snapshots in recovery. Sideload an incremental OTA on a Virtual A/B device in recovery is impossible and unsupported.
- This feature is enabled by default, and it can be disabled by changing the build flow.
- In build/make/target/product/virtual_ab_ota.mk, PRODUCT_VIRTUAL_AB_OTA and its property are assigned, so the virtual A/B OTA can be disabled by skipping this makefile and use legacy A/B update by removing the following line in device/mediatek/system/common/device.mk and device/mediatek/vendor/common/device.mk, and split build again.

```
$ (call inherit-product, $(SRC_TARGET_DIR)/product/virtual_ab_ota.mk)
```

1.7 Guideline for Generating A/B OTA Package

This section gives a description of a group of generating A/B System OTA package guideline. Starting with Android Q, MTK only supports split build by default. Please select the corresponding build package method according to the build method used by the project.

1.7.1 Generate A/B OTA Update Package by Split Build 1.0 (LD1.0) Step by Step

1.7.1.1 Full Build Project with Full otapackage.zip and targfiles.zip

1. Get split build 1.0 (LD1.0) command

- Execute the following command to obtain the complete split build command:

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${PROJ_NAME}-  
${eng|userdebug|user}
```

For example:

- Input command**

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_tb8786p1_64-  
userdebug
```

- Output command**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_t_64_cn-userdebug &&  
make sys_images
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_tb8786p1_64-userdebug && make  
vnd_images krn_images
```

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py --system-dir  
out_sys/target/product/mssi_t_64_cn/images --vendor-dir  
out/target/product/tb8786p1_64/images --kernel-dir out/target/product/tb8786p1_64/images --  
output-dir out/target/product/tb8786p1_64/merged
```

These three commands represent:

- Build system**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_t_64_cn-userdebug &&  
make sys_images
```

- Build vendor and kernel**

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_tb8786p1_64-userdebug && make  
vnd_images krn_image
```

- Merge**

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py  
--system-dir out_sys/target/product/mssi_t_64_cn/images  
--vendor-dir out/target/product/tb8786p1_64/images  
--kernel-dir out/target/product/tb8786p1_64/images
```

```
--output-dir out/target/product/tb8786p1_64/merged
```

2. Build full otapackage.zip

If you need to build full otapackage.zip, please add the '`--otapackage`' parameter to the merge command.

For example:

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py
--system-dir out_sys/target/product/mssi_t_64_cn/images
--vendor-dir out/target/product/tb8786p1_64/images
--kernel-dir out/target/product/tb8786p1_64/images
--output-dir out/target/product/tb8786p1_64/merged
--otapackage
```

OTA full package path:

E.g., out/target/product/\${PROJ}/merged/otapackage.zip

3. Build targetfiles.zip

If you need to build targetfiles.zip, please add the '`--targetfiles`' parameter to the merge command.

For example:

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py
--system-dir out_sys/target/product/mssi_t_64_cn/images
--vendor-dir out/target/product/tb8786p1_64/images
--kernel-dir out/target/product/tb8786p1_64/images
--output-dir out/target/product/tb8786p1_64/merged
--targetfiles
```

Target package path:

E.g., out/target/product/\${PROJ}/merged/ target_files.zip

1.7.1.2 Build Incremental Package

Define the current version as the **Source version** and the upgraded version as the **Target version**. Here are the steps to follow:

1. Follow the instructions in [Section 1.7.1.1 Full Build Project with Full otapackage.zip and targetfiles.zip](#) to perform a split build and obtain the targetfiles.zip for both the Source version and the Target version. Make sure that the compilation time of the Target version is later than that of the Source version.
2. Rename the targetfiles.zip of the Source version as "source.zip" and the targetfiles.zip of the Target version as "target.zip".
3. In the project compilation environment, execute the following commands sequentially:

Source & Lunch:

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_${SYS_RPOJ}-
${eng|userdebug|user}
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vnd_${PROJ}-${SYS_PROJ};
```

Centralize tools to out/host/linux-x86:

```
cp out_sys/host/linux-x86/* out/host/linux-x86/ -rf
```

```
cp out_sys/soong/host/linux-x86/* out/host/linux-x86/ -rf
```

```
cp out/soong/host/linux-x86/* out/host/linux-x86/ -rf
```

Build incremental package:**Android version ≤ Android T:**

```
python3 build/tools/releasetools/ota_from_target_files --block -v -p out/host/linux-x86 -k device/mediatek/common/security/releasekey -i source.zip target.zip delta.zip
```

Android version ≥ Android U:

```
./out/host/linux-x86/bin/ota_from_target_files -v -p out/host/linux-x86 -k device/mediatek/common/security/releasekey -i source.zip target.zip delta.zip
```

If your project do not have releasekey, please delete “-k device/mediatek/common/security/releasekey” in command, which will use default key (test key). Or use -k to specify the key path used by your project.

- **source.zip:** The targetfiles.zip file for the Source version.
- **target.zip:** The targetfiles.zip file for the Target version.
- **delta.zip:** The incremental package that upgrades from the Source version to the Target version.

Note: The specific naming of targetfiles.zip and incremental package is not restricted, and customers can set their own names as per their preference.

1.7.2 Generate A/B OTA Update Package by Split Build 2.0 (LD2.0) Step by Step

1.7.2.1 Full Build Project with Full otapackage.zip and targfiles.zip

1. Get split build 2.0 (LD2.0) command

Execute the following command to obtain the complete split build command:

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_${PROJ_NAME}-  
${eng|userdebug|user}
```

For example:

- **Input command**

```
./vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py full_  
auto8676p1_64_bsp-userdebug
```

- **Output command**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_auto_64_cn_armv82_car-  
userdebug && make sys_images
```

```
source build/envsetup.sh && export OUT_DIR=out_hal && lunch hal_mgvi_spm_64_armv82-userdebug
&& make hal_images
```

```
source build/envsetup.sh && export OUT_DIR=out_krn && lunch krn_mgk_64_k61-userdebug && make
krn_images
```

```
source build/envsetup.sh && export OUT_DIR=out && lunch vext_auto8676p1_64_bsp-userdebug &&
make vext_images
```

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images
--vendor-dir out_hal/target/product/mgvi_spm_64_armv82/images
--kernel-dir out_krn/target/product/mgk_64_k61/images
--vext-dir out/target/product/auto8676p1_64_bsp/images
--output-dir out/target/product/auto8676p1_64_bsp/merged
```

These five commands represent:

- **Build system**

```
source build/envsetup.sh && export OUT_DIR=out_sys && lunch sys_mssi_auto_64_cn_armv82_car-
userdebug && make sys_images
```

- **Build hal**

```
source build/envsetup.sh && export OUT_DIR=out_hal && lunch hal_mgvi_spm_64_armv82-userdebug
&& make hal_images
```

- **Build kernel**

```
source build/envsetup.sh && export OUT_DIR=out_krn && lunch krn_mgk_64_k61-userdebug && make
krn_images
```

- **Build vext**

```
source build/envsetup.sh && export OUT_DIR=out && lunch vext_auto8676p1_64_bsp-userdebug &&
make vext_images
```

- **Merge**

```
python out_sys/target/product/mssi_t_64_cn/images/split_build.py
--system-dir out_sys/target/product/mssi_t_64_cn/images
--vendor-dir out/target/product/tb8786p1_64/images
```

2. Build full otapackage.zip

If you need to build full otapackage.zip, please add the '**--otapackage**' parameter to the merge command.

For example:

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images
--vendor-dir out_hal/target/product/mgvi_spm_64_armv82/images
--kernel-dir out_krn/target/product/mgk_64_k61/images
--vext-dir out/target/product/auto8676p1_64_bsp/images
--output-dir out/target/product/auto8676p1_64_bsp/merged
--otapackage
```

Ota full package path:

E.g., out/target/product/\${PROJ}/merged/otapackage.zip

3. Build targetfiles.zip

If you need to build targetfiles.zip, please add the '`--targetfiles`' parameter to the merge command.

For example:

```
python out_sys/target/product/mssi_spm_64_cn_armv82/images/split_build.py
--system-dir out_sys/target/product/mssi_spm_64_cn_armv82/images
--vendor-dir out_hal/target/product/mgvi_spm_64_armv82/images
--kernel-dir out_krn/target/product/mgk_64_k61/images
--vext-dir out/target/product/auto8676p1_64_bsp/images
--output-dir out/target/product/auto8676p1_64_bsp/merged
--targetfiles
```

Target package path:

E.g., out/target/product/\${PROJ}/merged/ target_files.zip

1.7.2.2 Build Incremental Package

Please refer to the steps in Section [1.7.1.2 Build Incremental Package](#) to build an incremental package.

1.8 A/B System Updates Limitations

This section describes A/B system updates limitations.

This chapter is included from [\[2\]](#).

- OTA does not support modifying the partition table and changing the partition layout.
- Modifying the partition content of the current slot is not supported.
- Mutual upgrade between non-AB and AB is not supported.

1.9 How to Use and Debug A/B System Updates**1.9.1 How to Use A/B Updates**

- **Normal mode:**
 - PC must install python
 - Put update_device.py (system/update_engine/scripts/update_device.py) to adb folder
 - Connect phone with PC via USB cable
 - Python update_device.py update.zip in adb window

```
D:\adb>python update_device.py full_k53v1_64_ab-ota-1502713905.zip
INFO:root:Running: adb reverse tcp:1234 tcp:51501
INFO:root:Running: adb shell su 0 update_engine_client --update --follow
HASH=KyKz6JU+K3FQwYimMurD13xU0WGO2n3bESPb0t02sLM=
FILE_SIZE=592100123
METADATA_HASH=ytU6gL3h1dEzi9rnkWEeu1UXnIK1HSxTzAoA/fAEzD0=
METADATA_SIZE=86389
USER_AGENT=Dalvik <something. something>
NETWORK_ID=0
```

- **Recovery mode:**
 - ADB sideload update (VAB only supports full OTA):
 - adb reboot recovery
 - choose “Apply update form ADB” on recovery UI
 - adb sideload d:\update.zip
 - SD card update:
 - adb push update.zip /storage/F205-EFBE (/storage/F205-EFBE is SD card mount directory)
 - adb reboot recovery
 - choose “Apply update form SD card” on recovery UI
 - go on choosing the update.zip fi

1.9.2 How to Debug A/B Updates

Log location

- **Normal mode:**
 - Method one:
 - mtklog\mobilelog\APLogxxx\main_log
 - Search **update_engine** key word
 - Method Two:
 - adb pull data/misc/update_engine_log
 - Search **update_engine** key word
- **Recovery mode:**
 - Adb pull /tmp/recovery.log

Common error code

- All error code is in system/update_engine/common/error_code.h

1.10 Frequently Asked Questions

1.10.1 Does flashtool Need to Download Both A and B Images

- No, only the preloader partition needs to download the A and B images, and other partitions only download the A images by default.
- After flashtool downloads image, platform only runs on A images.
- A image could OTA to B empty partition through incremental update package

1.10.2 How to Know What Image is Running (A or B)

- Check property ro.boot.slot_suffix (getprop ro.boot.slot_suffix)
- Check androidboot.slot_suffix in command line (/proc/cmdline)

1.10.3 Could A Partition Load B Partition (e.g., lk_a loads boot_b)

- No, suffix must be the same. It's like dual system.

1.10.4 What Partition Needs A/B

- If partition needs OTA update, this partition must have both of A and B.

1.10.5 What's Feature Option for A/B System

- MTK_AB_OTA_UPDATER

1.10.6 If Handset Ships without A/B, Could Handset Become A/B through OTA Update

- No, AB and non-AB cannot upgrade each other.

1.10.7 Who Set boot_sucessfully Flag

- Update_verfier process will set boot_sucessfully flag.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.